neueda

# Lambda Expressions

# Objectives

- What is an Lambda?
- Simple Lambdas
- When to use Lambdas

# What are Lambdas?

- A lambda is a syntax allowing for the definition of a method with no name ie. anonymous method
- They make many tasks syntactically easier
  - Runnable methods in multithreading
  - Looping code
  - Filtering code
  - Event handling

# Basic Example

- Consider the following interface example

```
public interface Openable {
    void open();
}
```

- To create an implementation you could use

```
public class Shop implements Openable {
        public void open() {
                System.out.println(“open the shop”);
        }
}
```

- The type can then be instantiated

```
Openable someBuilding = new Shop();
```

# Using Lambdas

- Using a Lambda, the same code as the previous slide can be achieved with

```
public interface Openable {
    void open();
}
```

```
Openable someBuilding = () -> System.out.println(“open building”);
```

# Lambda Syntax

- The Lambda syntax is as follows

```
(any parameters) -> method body;
```

- So, in the example, the Openable interface has a single method that takes no parameters

```
() -> System.out.println("open building");
```

- The method body will require {} if there is more than one line of code

```
() -> {
    System.out.println("curlies required as ");
    System.out.prinlnt("more than one line");
};
```

# Lambda Parameters and Return Values

- Consider the following interface

```
public interface Maths {
    int add(int a, int b);
}
```

- The Lambda below could be used to create an implementation
  - The return is implied

```
Maths myMaths = (op1, op2) -> op1 + op2;
```

  - If you have a return statement, you need curly brackets

```
Maths myMaths = (op1, op2) -> {return op1 + op2;};
```

# Java 8 Lambda Interfaces

- Java 8 introduced some new interfaces specifically for Lambdas
  - Ideal where you have any of the following purposes

| Interface | Purpose |
|---|---|
| Function<S,T> | Takes a parameter of type S and returns a parameter of type T |
| Consumer<T> | Takes in a parameter of type T with no return value |
| Predicate<T> | Returns true or false based on a supplied type T |
| Supplier<T> | No parameters but returns an object of type T |

# Java 8 Lambda Interface Examples

```
// parameter and return value
Function<Integer, String> myFunc  = (a) -> {
                System.out.println(a);
                return "finished";
};
// returns true or false
Predicate<String> onlyReturnsTrueOrFalse = (s) -> true;
// takes a parameter no return value
Consumer<Integer> myCons = (a) -> System.out.println(a);
Consumer<Integer> myCons2 = System.out::println;
// returns a value but doesn't take a value
IntSupplier<Integer> mySupp = () -> {return 5+5;};
```

# Single Parameter Shorthand

- The **Method reference** is just compact and more readable form of a lambda expression for already existing methods

```
Consumer<Integer> myCons2 = System.out::println;
```

# When to Use a Lambda

- When you need the implementation of an interface ***when there is one abstract method to override***
- The examples shown only work because there is only one method in the interfaces

```
public interface Openable {
    void open();
}
```

```
public interface Maths {
    int add(int a, int b);
}
```

# Java 8 API Changes

- There have been many changes to the existing Java API to support Lambdas
- Commonly used changes are around
  - Threading
  - Collections
  - Sorting
  - Filtering
- You will be introduced to these as the course progresses

# Summary

- What is an Lambda?
- Simple Lambdas
- When to use Lambdas